

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
VIỆN TRÍ TUỆ NHÂN TẠO

BÁO CÁO MÔN HỌC KỸ THUẬT VÀ CÔNG NGHỆ DỮ LIỆU LỚN

ĐỀ TÀI

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG GỢI Ý
PHIM THỜI GIAN THỰC
TRÊN NỀN TẢNG DỮ LIỆU LỚN

Nhóm sinh viên thực hiện:

Lê Tuấn Anh – 22022622

HÀ NỘI – 2026

MỞ ĐẦU

Khi mở một ứng dụng xem phim, người dùng phải xem xét hàng chục nghìn lựa chọn khác nhau. Ít người có thời gian để xem qua toàn bộ kho nội dung đó, vì vậy các nền tảng như Netflix, Amazon hay YouTube đều đầu tư mạnh vào hệ thống gợi ý cá nhân hóa. Trên thực tế, phần lớn lượt xem trên những nền tảng này đến từ các đề xuất tự động thay vì từ thao tác tìm kiếm của người dùng.

Việc xây dựng một hệ thống gợi ý không dễ, đặc biệt khi dữ liệu lên tới hàng triệu người dùng và hàng triệu bộ phim. Trong báo cáo này, nhóm xây dựng một hệ thống gợi ý phim trên nền tảng dữ liệu lớn sử dụng bộ dữ liệu MovieLens 25M với khoảng 25 triệu lượt đánh giá. Hệ thống được thiết kế theo kiến trúc Lambda, kết hợp batch processing để huấn luyện mô hình trên toàn bộ dữ liệu lịch sử và xử lý theo luồng (stream processing) để cập nhật các thay đổi theo thời gian thực. Toàn bộ pipeline được triển khai với các công nghệ như Kafka, HDFS, Spark, HBase, MySQL, Airflow và Flask, đồng thời được đóng gói bằng Docker nhằm đơn giản hóa việc triển khai.

Thành phần chính của hệ thống là thuật toán ALS (Alternating Least Squares), một phương pháp phân rã ma trận phổ biến trong lọc cộng tác và được Spark MLlib hỗ trợ ở chế độ phân tán. Vì vậy mô hình có thể xử lý hiệu quả hàng chục triệu bản ghi. Điểm khác biệt của hệ thống là thay vì chỉ phát lại dữ liệu cũ để mô phỏng luồng thời gian thực, nhóm tích hợp thêm luồng sự kiện từ Wikimedia nhằm phản ánh những bộ phim đang thu hút sự quan tâm tại thời điểm hiện tại. Kết quả là một mô hình đạt sai số RMSE khoảng 0,80 cùng một ứng dụng web minh họa, cho phép người dùng nhập mã tài khoản để nhận danh sách phim.

Mục lục

MỞ ĐẦU	1
1 TỔNG QUAN	3
1.1 Giới thiệu	3
1.1.1 Tầm Quan trọng của bài toán gợi ý	3
1.1.2 Mục tiêu của dự án	3
1.1.3 Phạm vi dự án	4
2 PHƯƠNG PHÁP	5
2.1 Các công nghệ sử dụng	5
2.1.1 Apache Kafka	5
2.1.2 Hadoop HDFS	6
2.1.3 Apache Spark	6
2.1.4 Apache Airflow	7
2.1.5 Apache HBase	7
2.1.6 MySQL	7
2.1.7 Flask	8
2.1.8 Docker	8
2.1.9 Wikimedia EventStreams và TMDb	8
2.2 Thu thập và xử lý dữ liệu	8
2.2.1 Nguồn dữ liệu	8
2.2.2 Xử lý dữ liệu	9
2.3 Mô hình hóa và gợi ý	10
2.3.1 ALS hoạt động ra sao	10
2.3.2 Vì sao ALS hợp với bài toán này	10
2.3.3 Tham số và cách sinh gợi ý	11
2.4 Triển khai	11
2.4.1 Kiến trúc	11
2.4.2 Quy trình thực hiện	12
2.5 Kết quả	12
3 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	15

Chương 1

TỔNG QUAN

1.1 Giới thiệu

1.1.1 Tầm Quan trọng của bài toán gợi ý

Mục tiêu chính của hệ thống gợi ý là giúp người dùng vượt qua tình trạng quá tải thông tin trong các nền tảng có lượng nội dung khổng lồ. Thay vì phải tự tìm kiếm giữa hàng nghìn lựa chọn, người dùng có thể nhanh chóng tiếp cận những bộ phim phù hợp với sở thích cá nhân. Điều này không chỉ cải thiện trải nghiệm sử dụng mà còn giúp các doanh nghiệp tăng thời gian người dùng ở lại nền tảng, nâng cao mức độ tương tác và thúc đẩy doanh thu.

Việc xây dựng một hệ thống gợi ý hiệu quả rất nhiều khó khăn, đặc biệt khi dữ liệu có quy mô lớn và liên tục thay đổi:

- **Khối lượng dữ liệu lớn:** Hệ thống cần xử lý hàng chục triệu lượt tương tác giữa người dùng và phim. Khối lượng dữ liệu này vượt quá khả năng của một máy tính đơn lẻ, đòi hỏi các nền tảng lưu trữ và xử lý phân tán.
- **Tính thưa của dữ liệu:** Mỗi người dùng chỉ đánh giá hoặc xem một số lượng nhỏ phim trong toàn bộ kho dữ liệu. Điều này làm cho việc học mô hình và dự đoán trở nên khó khăn hơn.
- **Khó khăn khi xử lý người dùng mới hoặc phim mới:** Đối với người dùng mới hoặc phim mới chưa có lịch sử tương tác, hệ thống thiếu thông tin để đưa ra các gợi ý chính xác.
- **Sự thay đổi sở thích theo thời gian:** Sở thích của người dùng có thể thay đổi theo xu hướng hoặc nhu cầu cá nhân, vì vậy hệ thống cần được cập nhật thường xuyên để duy trì chất lượng gợi ý.

1.1.2 Mục tiêu của dự án

Mục tiêu của nhóm là xây dựng một hệ thống gợi ý phim trên nền tảng Big Data với các mục tiêu cụ thể như sau:

- Lưu trữ và xử lý bộ dữ liệu MovieLens 25M bằng HDFS và Spark nhằm đáp ứng yêu cầu xử lý dữ liệu quy mô lớn.
- Huấn luyện mô hình gợi ý sử dụng thuật toán ALS để tạo danh sách Top- N bộ phim phù hợp cho từng người dùng.
- Xây dựng hệ thống theo kiến trúc Lambda, kết hợp xử lý theo lô (batch processing) và xử lý theo luồng (stream processing) để đảm bảo gợi ý vừa chính xác vừa được cập nhật kịp thời.
- Tích hợp nguồn dữ liệu thời gian thực từ Wikimedia thay vì chỉ phát lại dữ liệu cũ, giúp hệ thống phản ánh các xu hướng mới đang diễn ra.

- Phát triển một ứng dụng web demo cho phép người dùng xem kết quả gợi ý một cách trực quan.
- Đóng gói toàn bộ hệ thống bằng Docker và sử dụng Airflow để tự động hóa việc huấn luyện cũng như cập nhật mô hình định kỳ.

1.1.3 Phạm vi dự án

Dự án bao gồm:

- Thu thập, lưu trữ và xử lý bộ dữ liệu MovieLens 25M trên HDFS kết hợp với Spark để hỗ trợ xử lý dữ liệu quy mô lớn.
- Xây dựng pipeline theo kiến trúc Lambda, sử dụng các công nghệ như Kafka, Spark, HBase và MySQL để đáp ứng cả xử lý theo lô và xử lý theo luồng. Tích hợp nguồn dữ liệu thời gian thực từ Wikimedia thay vì chỉ phát lại dữ liệu cũ
- Áp dụng thuật toán ALS (Alternating Least Squares) để huấn luyện mô hình và tạo danh sách gợi ý phim cho người dùng.
- Triển khai một ứng dụng web demo kèm REST API, đóng gói bằng Docker và sử dụng Airflow để điều phối các tác vụ cũng như tự động hóa quy trình xử lý.

Chương 2

PHƯƠNG PHÁP

2.1 Các công nghệ sử dụng

Hệ thống được ghép từ nhiều thành phần Big Data, mỗi thành phần đảm nhận một mảnh việc trong kiến trúc Lambda. Phần này trình bày chi tiết từng công cụ: bản chất, các thành phần cấu thành, kiến trúc/quy trình hoạt động, và vai trò cụ thể trong dự án.

2.1.1 Apache Kafka

Apache Kafka là một nền tảng truyền tin phân tán, được thiết kế để thu nhận và phân phối các luồng dữ liệu đến liên tục theo thời gian thực. Môi trường triển khai Kafka thường gồm các thành phần:

- **Zookeeper:** điều phối cluster - quản lý danh sách broker, lưu thông tin về topic và phân vùng (partition), đồng bộ trạng thái khi cụm thay đổi.
- **Broker:** là các server chịu trách nhiệm lưu trữ và phân phối dữ liệu. Mỗi broker giữ một phần partition (1 phần dữ liệu), và dữ liệu được sao chép giữa các broker để tăng độ sẵn sàng và khôi phục khi gặp sự cố.
- **Producer:** ứng dụng gửi message (thường là sự kiện) vào một topic cụ thể.
- **Consumer:** Là các ứng dụng hoặc thành phần gửi dữ liệu vào Kafka. Ứng dụng đọc message từ topic. Nhiều consumer có thể gộp thành một consumer group để chia sẻ tải xử lý.
- **Topic và Partition:** topic là kênh logic để gửi và nhận dữ liệu; mỗi topic được chia thành nhiều partition nhằm xử lý song song và nâng cao hiệu suất.
- **Kafka Connect:** công cụ kết nối Kafka với các hệ thống bên ngoài (cơ sở dữ liệu, HDFS, hệ thống messaging khác) mà không cần viết mã tích hợp thủ công.
- **Kafka Streams:** thư viện Java cho phép xử lý dữ liệu thời gian thực ngay bên trong Kafka.
- **Schema Registry:** quản lý và kiểm soát cấu trúc (schema) của message, giúp đảm bảo tính hợp lệ của dữ liệu trao đổi.

Nhờ kiến trúc phân tán, Kafka có thể xử lý hàng triệu message mỗi giây với độ trễ thấp và chịu lỗi tốt, nên rất phù hợp cho các ứng dụng cần phân phối dữ liệu thời gian thực.

Ứng dụng trong dự án: Kafka đóng vai trò Ingestion Layer. Mọi sự kiện đổ về topic `ratings-stream`, và điểm đặc biệt là topic này nhận đồng thời hai dòng dữ liệu khác nhau: luồng rating phát lại từ MovieLens (mô phỏng tương tác người dùng) và luồng phim đang được chỉnh sửa lấy thật từ Wikimedia. Consumer ở tầng sau sẽ tự phân loại hai dòng này để xử lý theo cách riêng.

2.1.2 Hadoop HDFS

HDFS là hệ thống tệp phân tán, được thiết kế để chạy trên phần cứng thương mại thông thường và tối ưu cho việc lưu trữ dữ liệu lớn.

a) Đặc điểm chính:

- **Lưu trữ phân tán:** dữ liệu được chia thành các khối (block, mặc định 128/256 MB) và phân phối trên nhiều node trong cụm.
- **Chịu lỗi cao:** mỗi block được nhân bản (mặc định 3 bản sao) trên các node khác nhau, nên hệ thống vẫn hoạt động bình thường ngay cả khi một số node gặp sự cố.
- **Thiết kế cho dữ liệu lớn:** tối ưu cho tệp lớn (hàng trăm GB đến vài PB) theo mô hình ghi một lần, đọc nhiều lần, phù hợp các ứng dụng phân tích.
- **Khả năng mở rộng và chi phí thấp:** có thể thêm node mới mà không gián đoạn dịch vụ, và chạy được trên phần cứng phổ thông.

b) **Kiến trúc:** HDFS hoạt động theo mô hình Client-Server với hai thành phần chính. **NameNode** là nút quản lý trung tâm, giữ metadata (cấu trúc thư mục, danh sách block và vị trí lưu trữ) nhưng không chứa dữ liệu thực. **DataNode** là nơi lưu các block dữ liệu thực tế và thực hiện đọc/ghi theo yêu cầu. Ngoài ra còn có **Secondary NameNode** hỗ trợ lưu bản sao metadata định kỳ để giảm tải cho NameNode.

c) **Quy trình hoạt động:** khi ghi, tệp được cắt thành các block và phân tới nhiều DataNode, NameNode ghi lại vị trí từng block. Khi đọc, client hỏi NameNode để biết vị trí block, rồi truy cập trực tiếp các DataNode. Nếu một DataNode hỏng, NameNode chỉ định bản sao trên DataNode khác để phục hồi.

d) **Ứng dụng trong dự án:** HDFS là Data Lake lưu bộ MovieLens 25M thô - `ratings.csv` cỡ 647 MB và `movies.csv` - để Spark đọc song song khi huấn luyện ALS, đồng thời đảm bảo chịu lỗi cho dữ liệu lịch sử.

2.1.3 Apache Spark

Apache Spark là engine xử lý dữ liệu lớn trong bộ nhớ, nhanh hơn nhiều lần so với MapReduce truyền thống. Môi trường Spark gồm các thành phần:

- **Driver:** nơi khởi tạo và điều khiển ứng dụng, gửi job tới cluster và thu kết quả.
- **Cluster Manager:** quản lý tài nguyên và phân phối công việc cho các worker (có thể là Standalone, YARN, Mesos hoặc Kubernetes).
- **Worker và Executor:** mỗi worker chạy một hoặc nhiều executor - tiến trình thực thi task, tính toán và cache dữ liệu tạm.
- **RDD / DataFrame:** cấu trúc dữ liệu phân tán, chịu lỗi, hỗ trợ các phép biến đổi song song như map, reduce, filter.
- **Các thư viện đi kèm:** Spark SQL (truy vấn dạng bảng), Spark Streaming (Cho phép xử lý dữ liệu thời gian thực bằng cách xử lý theo micro-batch), **MLlib** (học máy phân tán), GraphX (xử lý đồ thị).

Spark tích hợp tốt với hệ sinh thái Hadoop, đọc/ghi được HDFS, HBase và nhiều nguồn khác.

Ứng dụng trong dự án: nhóm dùng **PySpark** cùng thư viện **MLlib** để huấn luyện mô hình ALS phân tán trên toàn bộ 25 triệu bản ghi: đọc dữ liệu từ HDFS, huấn luyện, đánh giá RMSE/MAE, sinh Top-N gợi ý và ghi kết quả ra MySQL.

2.1.4 Apache Airflow

Apache Airflow là nền tảng mã nguồn mở dùng để lập lịch, giám sát và quản lý workflow dưới dạng Directed Acyclic Graph viết bằng Python.

a) Đặc điểm chính: workflow định nghĩa bằng mã Python nên dễ mô hình hóa ETL hay huấn luyện mô hình; có khả năng mở rộng trên nhiều worker; tích hợp sẵn với Hadoop, Spark, HDFS, Kafka, MySQL và cung cấp giao diện web trực quan để theo dõi tiến trình, khởi chạy workflow và xem log.

b) Kiến trúc: gồm **Scheduler** (lập lịch và phân phối task theo DAG), **Executor** (thực thi task - Local/Celery/Kubernetes Executor), **Web Server** (giao diện quản lý) và **Metadata Database** (lưu trạng thái DAG, task và lịch sử chạy).

c) Ứng dụng trong dự án: Airflow điều phối nhánh batch qua DAG `movielens_recsys_batch`: kiểm tra dữ liệu MovieLens đã có trên HDFS, gọi Spark huấn luyện ALS, rồi xác nhận Batch View trong MySQL đã sẵn sàng. Lịch đặt theo ngày nên mô hình được làm mới đều đặn; khi một task lỗi, Airflow tự động thử lại.

2.1.5 Apache HBase

HBase là cơ sở dữ liệu NoSQL hướng cột, chạy trên HDFS, hỗ trợ đọc và ghi ngẫu nhiên với độ trễ thấp - phù hợp cho dữ liệu cần truy vấn nhanh theo thời gian thực. Các thành phần chính:

- **HMaster:** điều phối, quản lý và giám sát các RegionServer, phân phối region và theo dõi trạng thái cụm.
- **RegionServer:** lưu và phục vụ một hoặc nhiều region, thực hiện thao tác đọc và ghi.
- **Region:** mỗi region chứa một phần của bảng và được phân phối/cân bằng lại giữa các RegionServer khi cần.
- **Zookeeper:** quản lý cấu hình và theo dõi trạng thái các RegionServer trong cụm.
- **HBase Table:** tổ chức theo kiểu keyvalue với các column family, mở rộng theo chiều ngang bằng cách thêm RegionServer.
- **HFile và MemStore:** dữ liệu ghi vào MemStore (bộ nhớ tạm) trước, rồi được đẩy xuống HFile trên đĩa khi đạt ngưỡng kích thước.
- **Client API:** truy cập qua Java, REST hoặc **Thrift** - dự án dùng Thrift thông qua thư viện `happybase`.

HBase thường được triển khai cùng Hadoop để tận dụng HDFS cho lưu trữ phân tán, đảm bảo tính sẵn sàng cao và khả năng mở rộng.

Ứng dụng trong dự án: HBase là **Real-time View (Speed Layer)**, lưu hai loại dữ liệu theo khóa dòng: gợi ý real-time của từng người dùng (khóa `userId`) và danh sách phim đang thịnh hành (khóa `__TRENDING__`). Cả hai được tăng luồng cập nhật liên tục và web truy vấn nhanh để hiển thị.

2.1.6 MySQL

MySQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, mạnh mẽ và tuân thủ chuẩn SQL.

a) Đặc điểm chính: hỗ trợ giao dịch **ACID** (Atomicity, Consistency, Isolation, Durability) đảm bảo toàn vẹn dữ liệu; đánh chỉ mục (index) hiệu quả như B-Tree giúp truy vấn nhanh; mở rộng tốt và có cộng đồng lớn, được cập nhật liên tục.

b) Ứng dụng trong dự án: MySQL đóng vai trò **Batch View**, lưu kết quả đã xử lý từ tầng batch: bảng `movies` (metadata phim, kèm cột `poster_url`), bảng gợi ý `user_recommendations` do ALS sinh

ra, bảng phim phổ biến `popular_movies` và bảng thống kê `stats`. Tầng Serving (Flask) truy vấn trực tiếp các bảng này.

2.1.7 Flask

Flask là một micro-framework web viết bằng Python, nhẹ và linh hoạt.

a) **Đặc điểm chính:** chỉ cung cấp các thành phần cốt lõi, cho phép thêm module khi cần; hỗ trợ xây dựng **RESTful API** dễ dàng; tích hợp tốt với các thư viện Python như SQLAlchemy; và dùng template Jinja2 để dựng giao diện động.

b) **Ứng dụng trong dự án:** Flask là Serving Layer - trang web demo cho phép nhập User ID và nhận danh sách phim gợi ý (kèm poster), đồng thời mở các REST API `/api/recommend/<user_id>` và `/api/trending` trả dữ liệu JSON, cùng một dashboard thống kê tổng quan.

2.1.8 Docker

Docker là nền tảng đóng gói ứng dụng vào các container - môi trường cô lập chứa đầy đủ thư viện và công cụ để chạy độc lập với hệ thống host.

a) **Đặc điểm chính:** cô lập môi trường, đảm bảo tính nhất quán giữa phát triển, kiểm thử và triển khai; container nhẹ hơn máy ảo nên khởi động nhanh và ít tốn tài nguyên; dễ di động sang mọi hệ thống hỗ trợ Docker; và đặc biệt, Docker Compose cho phép định nghĩa, quản lý nhiều container như một hệ thống thống nhất.

b) **Ứng dụng trong dự án:** toàn bộ 13 container - Kafka, Zookeeper, HDFS (NameNode, DataNode), Spark (master, worker), HBase, MySQL, Airflow, hai producer (replay rating và Wikimedia), consumer, và Flask — được đóng gói và khởi chạy đồng thời bằng đúng một lệnh `docker compose up`, đảm bảo tái lập hệ thống dễ dàng.

2.1.9 Wikimedia EventStreams và TMDB

Ngoài hệ sinh thái Big Data lõi, dự án dùng thêm hai nguồn bên ngoài. **Wikimedia EventStreams** là một luồng SSE (Server-Sent Events) công khai, phát trực tiếp mọi chỉnh sửa trên Wikipedia theo thời gian thực, miễn phí và không cần khóa API. Nhóm dùng nó như một cảm biến để biết phim nào đang được người ta chỉnh sửa - tức là đang được chú ý. **TMDB (The Movie Database)** cung cấp poster thật cho từng phim thông qua `tmdbId` có sẵn trong `links.csv`, giúp giao diện trông giống một ứng dụng xem phim thực thụ thay vì một bảng dữ liệu.

2.2 Thu thập và xử lý dữ liệu

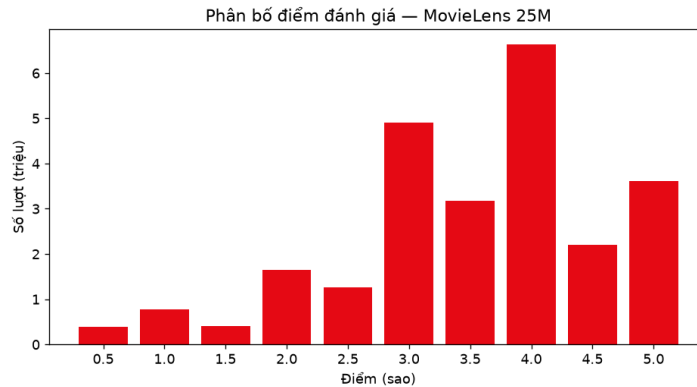
2.2.1 Nguồn dữ liệu

Hệ thống chạy trên ba nhóm dữ liệu. Một là bộ MovieLens 25M của GroupLens.

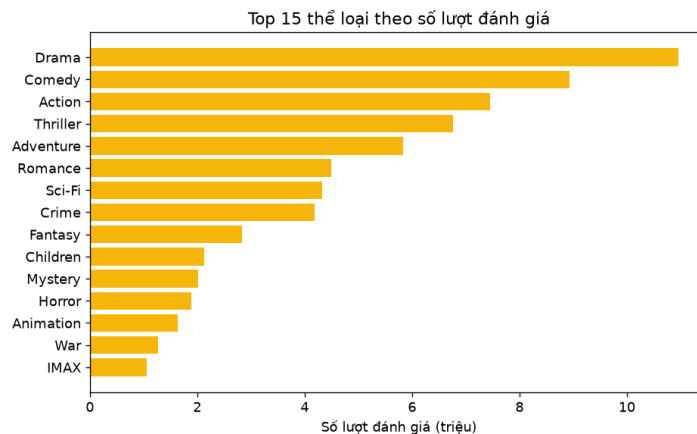
Tệp	Mô tả	Kích thước
<code>ratings.csv</code>	25.000.095 lượt đánh giá: <code>userId</code> , <code>movieId</code> , <code>rating</code> , <code>timestamp</code>	≈647 MB
<code>movies.csv</code>	62.423 phim: <code>movieId</code> , <code>title</code> , <code>genres</code>	≈3 MB
<code>links.csv</code>	Liên kết IMDB/TMDB, dùng để lấy poster	≈1.4 MB
<code>tags.csv</code>	≈1 triệu tag do người dùng gán	≈37 MB

Thang điểm chạy từ 0.5 đến 5.0 theo bước 0.5, với khoảng 162.000 người dùng. Nhìn vào phân bố điểm (Hình 2.1) sẽ thấy một thói quen: đa số dồn về 3.0–4.0, ít ai chấm quá thấp. Về thể loại (Hình 2.2), Drama, Comedy và Action chiếm phần lớn lượt đánh giá.

Nhóm dữ liệu thứ hai là luồng thời gian thực từ Wikimedia. Đây là tín hiệu thời gian thực: nhóm lọc các trang phim đang được sửa trên Wikipedia để suy ra phim đang được quan tâm, rồi đưa vào tăng tốc độ. Nhóm thứ ba, là poster từ TMDB - nhóm đã tải về cho gần 12.000 phim phổ biến nhất và lưu vào cột `movies.poster_url`.



Hình 2.1: Phân bố điểm đánh giá của MovieLens 25M.



Hình 2.2: Top thể loại theo số lượt đánh giá.

2.2.2 Xử lý dữ liệu

Quá trình tiền xử lý dữ liệu được thực hiện thông qua một chuỗi các bước liên tục nhằm bảo đảm dữ liệu đầu vào đáp ứng yêu cầu của hệ thống gợi ý. Trước tiên, bộ dữ liệu MovieLens được nạp lên HDFS bằng script `scripts/load_to_hdfs.sh`. Song song với đó, hệ thống sử dụng hai tiến trình producer để đưa dữ liệu vào Kafka. Trong đó, một producer phát lại các bản ghi đánh giá (rating) từ MovieLens để mô phỏng luồng dữ liệu thời gian thực, còn producer `wiki_stream_producer.py` thu thập trực tiếp các sự kiện thay đổi từ Wikimedia.

Ở giai đoạn làm sạch dữ liệu, các trường `userId` và `movieId` được chuyển về kiểu số nguyên, trong khi trường `rating` được ép sang kiểu số thực nhằm đảm bảo tính nhất quán trong quá trình xử lý. Những bản ghi thiếu thông tin hoặc chứa giá trị không hợp lệ được loại bỏ để tránh ảnh hưởng đến chất lượng

của mô hình. Đối với bài toán cold-start, tức các người dùng hoặc bộ phim chưa từng xuất hiện trong tập huấn luyện, mô hình ALS được cấu hình với tùy chọn `coldStartStrategy="drop"` để loại bỏ các giá trị dự đoán không hợp lệ ('NaN') trong quá trình đánh giá.

Sau khi hoàn tất bước làm sạch, hệ thống tiếp tục trích xuất các đặc trưng phục vụ việc gợi ý. Cụ thể, điểm đánh giá trung bình và số lượt đánh giá của từng bộ phim được tính toán, năm phát hành được tách từ tiêu đề thông qua biểu thức chính quy, đồng thời danh sách thể loại được phân tích nhằm hỗ trợ chức năng gợi ý theo thể loại trong tầng xử lý thời gian thực. Để xác định các bộ phim phổ biến, nhóm sử dụng chỉ số **weighted rating** theo công thức của IMDB thay vì chỉ dựa trên điểm trung bình. Cái này giúp hạn chế tình trạng những bộ phim có rất ít lượt đánh giá nhưng vẫn đạt điểm trung bình cao và được xếp hạng thiếu hợp lý.

Bên cạnh dữ liệu MovieLens, hệ thống còn thực hiện chuẩn hóa tiêu đề phim nhằm thống nhất cách biểu diễn giữa MovieLens và Wikipedia, qua đó nâng cao khả năng ghép nối chính xác dữ liệu trong quá trình xử lý các sự kiện thời gian thực. Cuối cùng, toàn bộ dữ liệu được chia ngẫu nhiên thành tập huấn luyện và tập kiểm thử theo tỷ lệ 80/20. Giá trị **seed** được cố định để bảo đảm các thí nghiệm có thể được tái lập và cho kết quả nhất quán.

2.3 Mô hình hóa và gợi ý

Thuật toán sẽ sử dụng của hệ thống là ALS (Alternating Least Squares), một phương pháp lọc cộng tác dựa trên kỹ thuật phân rã ma trận để học sở thích của người dùng và đặc trưng của phim từ dữ liệu đánh giá, từ đó dự đoán những bộ phim mà người dùng có thể yêu thích.

2.3.1 ALS hoạt động ra sao

Có thể hình dung dữ liệu đánh giá dưới dạng một ma trận R , trong đó mỗi hàng tương ứng với một người dùng và mỗi cột tương ứng với một bộ phim. Mỗi ô trong ma trận biểu thị điểm mà người dùng dành cho bộ phim đó. Tuy nhiên, trên thực tế hầu hết các ô đều để trống vì mỗi người dùng chỉ xem và đánh giá một số ít phim. Thuật toán ALS giải quyết vấn đề này bằng cách xấp xỉ ma trận này bằng tích của hai ma trận nhỏ hơn, $R \approx UV^T$, trong đó mỗi người dùng được mô tả bằng một vector k chiều trong U và mỗi phim bằng một vector k chiều trong V . phản ánh các đặc trưng tiềm ẩn mà mô hình tự học được từ dữ liệu. Những đặc trưng này có thể được hiểu như sở thích về thể loại hoặc phong cách nội dung (ví dụ: hành động, lãng mạn hay hài hước), mặc dù mô hình không gán tên cụ thể cho từng chiều.. Nhân hai vector lại sẽ ra điểm dự đoán cho một cặp người dùng-phim. Bài toán cần tối thiểu hóa là:

$$\min_{U,V} \sum_{(u,i) \in \mathcal{O}} (r_{ui} - u_u^T v_i)^2 + \lambda \left(\sum_u \|u_u\|^2 + \sum_i \|v_i\|^2 \right),$$

với $\mathcal{O}$ là tập cặp người dùng-phim đã có đánh giá và λ là hệ số điều chuẩn. Điểm đặc biệt của ALS là nằm ở chữ “alternating”. Thay vì tối ưu đồng thời cả U lẫn V là bài toán không lồi và khó, thuật toán sẽ cố định một ma trận để tìm ma trận còn lại bằng phương pháp bình phương tối thiểu (có nghiệm đóng), sau đó đổi vai hai ma trận và tiếp tục lặp lại quá trình này cho đến khi hội tụ. Nhờ mỗi hàng được giải độc lập, bài toán có thể được chia nhỏ và thực hiện song song trên Spark một cách hiệu quả..

2.3.2 Vì sao ALS hợp với bài toán này

ALS được lựa chọn vì phù hợp với đặc điểm của bài toán gợi ý trên dữ liệu lớn. Trước hết, thuật toán có khả năng song song hóa một cách tự nhiên nên dễ dàng mở rộng khi khối lượng dữ liệu tăng lên. Bên cạnh đó, tham số **regParam** đóng vai trò điều chuẩn, giúp hạn chế overfitting trên ma trận đánh giá thưa.

Tham số `coldStartStrategy` xử lý gọn trường hợp người dùng hay phim lạ. Và nếu sau muốn dùng tín hiệu ngầm như lượt xem thay cho điểm số, ALS có sẵn tùy chọn `implicitPrefs` để chuyển.

2.3.3 Tham số và cách sinh gợi ý

Sau khi thử nghiệm, nhóm chốt bộ tham số trong bảng dưới. Quy trình huấn luyện như sau: đọc ratings từ HDFS, chia 80/20, dựng mô hình ALS, lặp cập nhật U và V qua `maxIter` vòng, rồi đo RMSE và MAE trên tập kiểm thử bằng `RegressionEvaluator`.

Tham số	Giá trị	Ý nghĩa
<code>rank (k)</code>	64	Số nhân tố ẩn
<code>maxIter</code>	10	Số vòng lặp ALS
<code>regParam (λ)</code>	0.08	Hệ số điều chuẩn
<code>coldStartStrategy</code>	drop	Xử lý user/item mới
Top-N	20	Số phim gợi ý mỗi user
MIN_REC_RATINGS	1000	Chỉ gợi ý phim có ≥ 1000 lượt

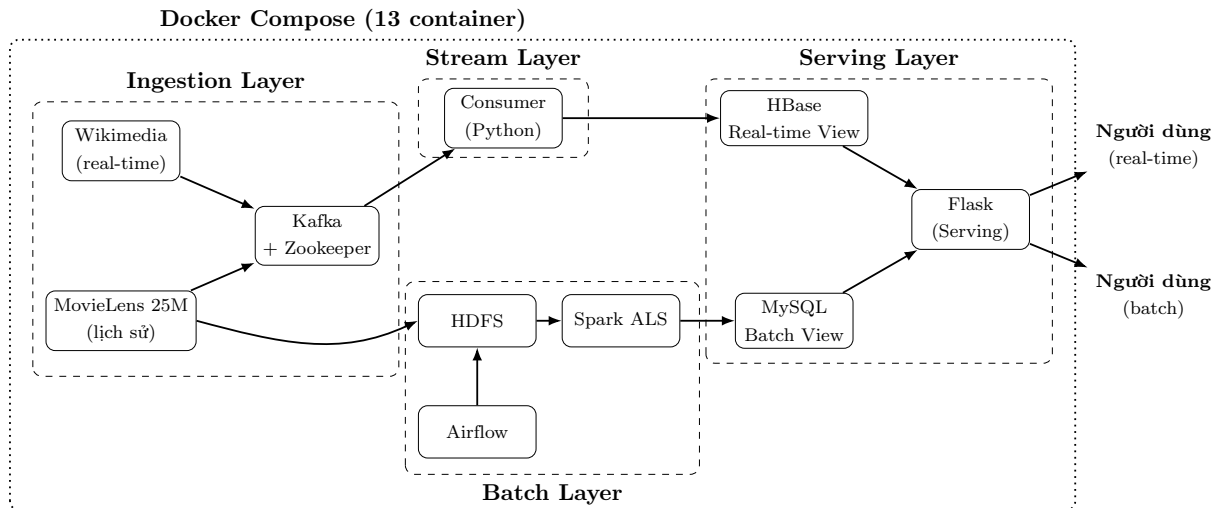
Để lấy gợi ý, nhóm gọi `recommendForAllUsers` sinh Top-N cho từng người, chuyển kết quả thành các dòng (`user_id`, `rank`, `movie_id`, `score`) rồi ghi vào MySQL. Chỉ giữ phim có từ 1.000 lượt đánh giá trở lên. Không có điều đó, ALS đôi khi đẩy lên đầu vài phim quá hiếm với điểm cao bất thường, khiến gợi ý mất tin cậy. Người dùng mới, vốn chưa có gì để dựa vào, được phục vụ bằng bảng phim phổ biến.

2.4 Triển khai

2.4.1 Kiến trúc

- Hệ thống được xây dựng theo kiến trúc Lambda gồm ba tầng, như minh họa trong Hình 2.3. Tầng batch đảm nhiệm các tác vụ xử lý chính, trong đó dữ liệu MovieLens 25M được lưu trữ trên HDFS, Spark huấn luyện ALS và sinh ra danh sách gợi ý Top-N, sau đó kết quả được lưu vào MySQL, và Airflow lo việc làm mới định kỳ.
- Tầng tốc độ(speed layer) tập trung xử lý dữ liệu thời gian thực. Hệ thống tiếp nhận hai luồng dữ liệu thông qua Kafka: luồng sự kiện từ Wikimedia để xác định các bộ phim đang thịnh hành và luồng replay rating từ MovieLens để hỗ trợ gợi ý theo thể loại. Các consumer tiếp nhận và xử lý dữ liệu ngay khi phát sinh, sau đó ghi kết quả vào HBase với độ trễ thấp.
- Cuối cùng, tầng phục vụ(serving layer) được triển khai bằng Flask, đóng vai trò kết hợp dữ liệu từ cả hai tầng trên. Các gợi ý được tính toán trước từ MySQL và các gợi ý cập nhật theo thời gian thực từ HBase được tổng hợp, bổ sung thêm poster từ TMDB, rồi trả về cho người dùng thông qua giao diện web và API.

Kiến trúc này mang lại ba lợi ích chính. Thứ nhất, hệ thống có khả năng mở rộng tốt nhờ các thành phần như Kafka, Spark và HDFS đều được thiết kế để xử lý dữ liệu với quy mô lớn. Thứ hai, kiến trúc Lambda giúp cân bằng giữa độ chính xác và tính cập nhật của kết quả gợi ý: mô hình được huấn luyện trên toàn bộ dữ liệu lịch sử để tạo ra các đề xuất có độ chính xác cao, trong khi tầng xử lý thời gian thực liên tục cập nhật các xu hướng và sự kiện mới để đảm bảo kết quả luôn kịp thời. Cuối cùng, hệ thống có tính linh hoạt cao, cho phép thay đổi thuật toán, mở rộng năng lực lưu trữ hoặc nâng cấp từng thành phần riêng lẻ mà không cần thiết kế lại toàn bộ kiến trúc.



Hình 2.3: Kiến trúc Lambda của hệ thống gợi ý phim.

2.4.2 Quy trình thực hiện

Trên thực tế, chạy hệ thống đi qua mấy bước theo thứ tự. Đầu tiên, `docker compose up` dựng 13 container. Tiếp theo, `load_to_hdfs.sh` đưa MovieLens lên HDFS và MySQL khởi tạo schema từ `mysql_init.sql`. Sau đó là nhánh batch: gửi `train_als.py` lên Spark để huấn luyện, đánh giá và ghi Batch View. Nhánh luồng chạy song song, với `wiki_stream_producer.py` bơm sự kiện phim live vào Kafka và `consumer.py` cập nhật HBase. Một bước làm giàu chèn vào giữa: `fetch_posters.py` kéo poster từ TMDB về MySQL. Cuối cùng Flask phục vụ demo ở `http://localhost:5000`, còn Airflow lo lịch huấn luyện lại.

2.5 Kết quả

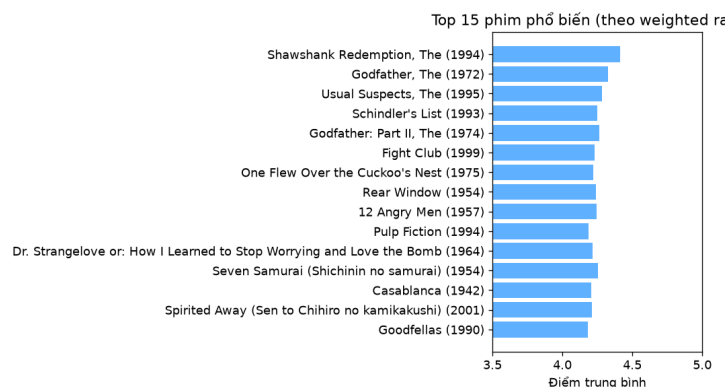
Thí nghiệm chạy ALS với `rank=64`, `maxIter=10`, `regParam=0.08` trên toàn bộ 25.000.095 lượt đánh giá, chia 80/20. Bảng dưới tóm tắt quy mô dữ liệu và độ chính xác.

Chỉ số	Giá trị
Tổng số lượt đánh giá	25.000.095
Số người dùng	≈162.541
Số phim	62.423
Tập huấn luyện / kiểm thử	80% / 20%
RMSE	0.7965
MAE	0.6187

Trên thang 0.5–5.0, RMSE khoảng 0.80 là kết quả tốt. Để dễ hình dung, các hệ phân rã ma trận công bố trên MovieLens thường rơi vào quãng 0.80–0.87, nên mô hình của nhóm nằm ở mép trên của nhóm này. MAE khoảng 0.62 còn trực quan hơn: trung bình mỗi dự đoán chỉ lệch khoảng 0.62 sao so với điểm thật.

Từ mô hình, hệ thống sinh Top-20 cho mỗi người dùng, tổng cộng 400.000 dòng gợi ý cho 20.000 người dùng mẫu, sau khi đã lọc bỏ phim dưới 1.000 lượt đánh giá. Bảng phim phổ biến (Top-100 theo weighted rating, Hình 2.4) dành cho người dùng mới. Lấy người dùng số 100 làm ví dụ, gợi ý trả về gồm *Planet Earth II*, *The Lives of Others*, *12 Angry Men*, *Casablanca*, *Witness for the Prosecution*... với điểm dự

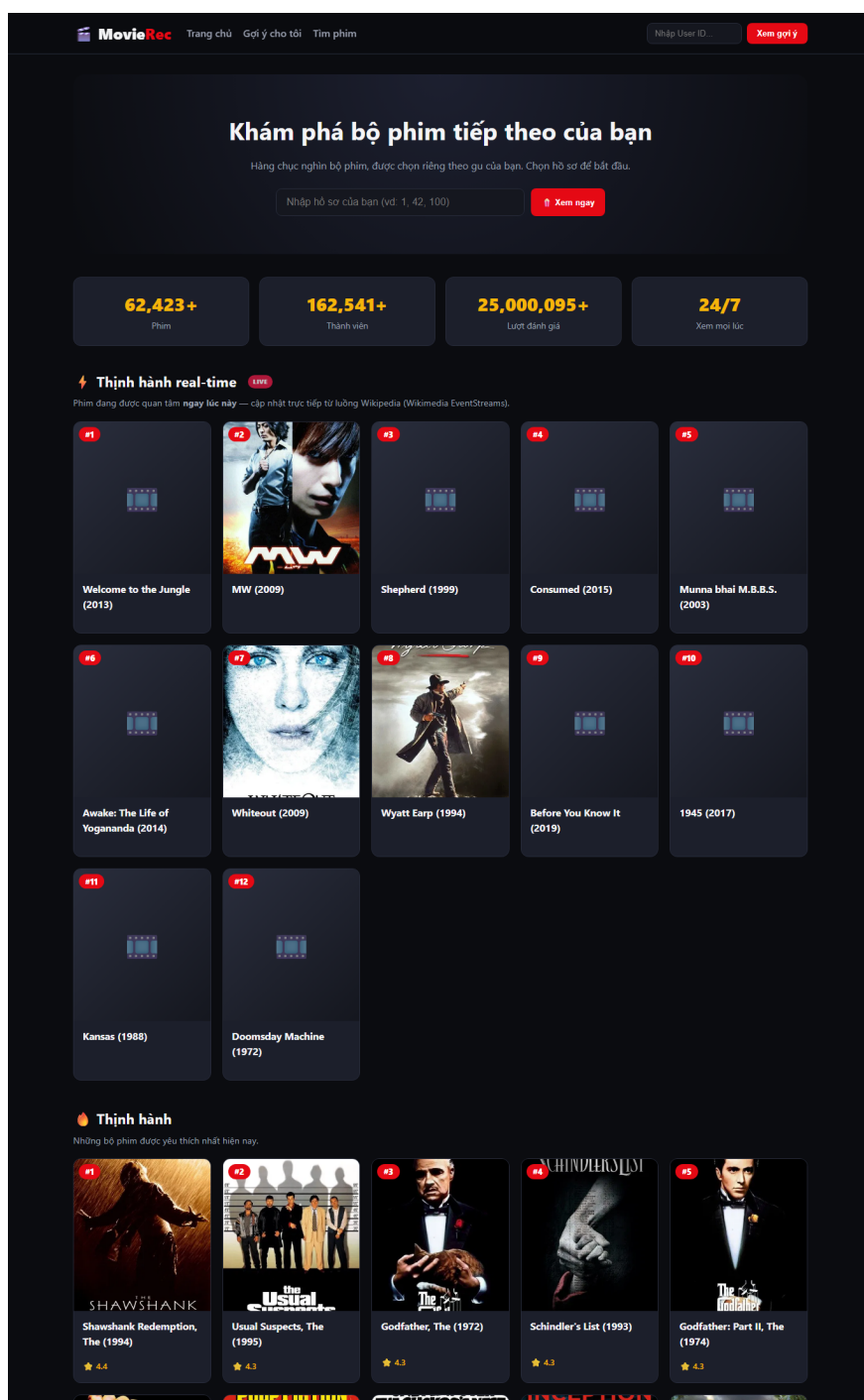
đoán quanh 4.2–4.4 — toàn phim kinh điển, cho thấy mô hình bắt được gu phim chất lượng cao của người này.



Hình 2.4: Top phim phổ biến theo weighted rating.

Phần nổi bật nhất của hệ thống là khả năng xử lý dữ liệu theo thời gian thực. Producer Wikimedia duy trì kết nối với luồng SSE và liên tục nhận các sự kiện chỉnh sửa từ Wikipedia trên toàn thế giới. Sau khi lọc dữ liệu và đối chiếu tên phim, hệ thống nhận diện được trung bình khoảng 2 phim mỗi phút đang thu hút sự chú ý của cộng đồng. Điều quan trọng là đây là dữ liệu trực tiếp (live data), không phải dữ liệu phát lại hay được tạo sẵn

Trong một lần chạy thực tế, mục “Thịnh hành real-time” trên trang chủ đã hiển thị nhiều bộ phim vừa được người dùng Wikipedia chỉnh sửa, chẳng hạn như Welcome to the Jungle, Wyatt Earp, Munna Bhai M.B.B.S. và Whiteout. Kết quả này cho thấy hệ thống có thể phản ánh khá nhanh những nội dung đang được cộng đồng quan tâm tại thời điểm hiện tại.



Hình 2.5: Danh sách phim thịnh hành real-time sinh từ luồng Wikimedia EventStreams (chụp trực tiếp từ hệ thống đang chạy).

Chương 3

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Nhóm đã xây dựng thành công một hệ thống gợi ý phim hoàn chỉnh trên nền tảng Big Data, sử dụng kiến trúc Lambda và lấy thuật toán ALS làm mô hình cốt lõi. Hệ thống không chỉ tạo ra các gợi ý dựa trên dữ liệu lịch sử với độ chính xác cao mà còn có khả năng cập nhật các bộ phim đang thịnh hành theo thời gian thực từ nguồn dữ liệu trực tiếp. Toàn bộ kết quả được hiển thị thông qua giao diện web demo có tích hợp poster phim, giúp người dùng dễ dàng theo dõi và trải nghiệm.

Về kết quả thực nghiệm, mô hình đạt RMSE là 0.7965 và MAE là 0.6187, đồng thời tạo ra khoảng 400.000 gợi ý cho người dùng. Về mặt triển khai, hệ thống gồm 13 container hoạt động đồng bộ và đã được kiểm thử thành công trên toàn bộ quy trình từ thu thập dữ liệu, xử lý, huấn luyện mô hình đến cung cấp kết quả gợi ý. Ngoài ra, kiến trúc và mã nguồn của hệ thống có tính tổng quát cao, nên có thể được tái sử dụng cho nhiều bài toán gợi ý khác như thương mại điện tử, nền tảng nghe nhạc hoặc hệ thống đề xuất tin tức chỉ bằng cách thay đổi nguồn dữ liệu tương tác.

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế. Hiện tại, mô hình sử dụng ALS thuần túy và chưa kết hợp với các phương pháp gợi ý dựa trên nội dung (content-based) hoặc các mô hình học sâu để cải thiện chất lượng đề xuất. Hệ thống cũng mới được triển khai dưới dạng mô phỏng phân tán trên một máy tính, vì vậy khả năng hoạt động trên cụm nhiều nút thực tế vẫn chưa được kiểm chứng. Ngoài ra, dữ liệu thu thập từ Wikimedia có tần suất tương đối thấp, trung bình chỉ khoảng hai bộ phim mỗi phút, nên cần một khoảng thời gian nhất định để hình thành danh sách phim thịnh hành. Việc đánh giá mô hình hiện chủ yếu dựa trên RMSE và MAE, trong khi các chỉ số quan trọng đối với bài toán gợi ý như Precision@K, Recall@K hay NDCG vẫn chưa được xem xét.

Từ các hạn chế trên, có thể phát triển một số hướng trong tương lai. Hệ thống có thể kết hợp ALS với phương pháp content-based dựa trên thể loại hoặc tag của phim nhằm giảm ảnh hưởng của bài toán cold-start. Đồng thời, việc thử nghiệm các mô hình học sâu như Neural Collaborative Filtering (Neural CF) hoặc kiến trúc Two-Tower cũng là một hướng để nâng cao chất lượng gợi ý. Bên cạnh đó, bổ sung các chỉ số đánh giá như Precision@K, Recall@K và NDCG sẽ giúp đánh giá toàn diện hơn hiệu quả xếp hạng của hệ thống. Đối với phần demo, có thể tích hợp chức năng cho phép người dùng đánh giá phim trực tiếp trên giao diện web để tạo ra các sự kiện thời gian thực, thay vì chỉ phụ thuộc vào dữ liệu từ Wikimedia.

Tài liệu tham khảo

- [1] Y. Koren, R. Bell, and C. Volinsky. “Matrix Factorization Techniques for Recommender Systems.” *IEEE Computer*, 2009.
- [2] F. M. Harper and J. A. Konstan. “The MovieLens Datasets: History and Context.” *ACM TiiS*, 2015.
- [3] Apache Spark MLlib — Collaborative Filtering (ALS). <https://spark.apache.org/docs/latest/ml-collaborative-filtering.html>
- [4] N. Marz and J. Warren. *Big Data: Principles and best practices of scalable real-time data systems*. Manning, 2015.